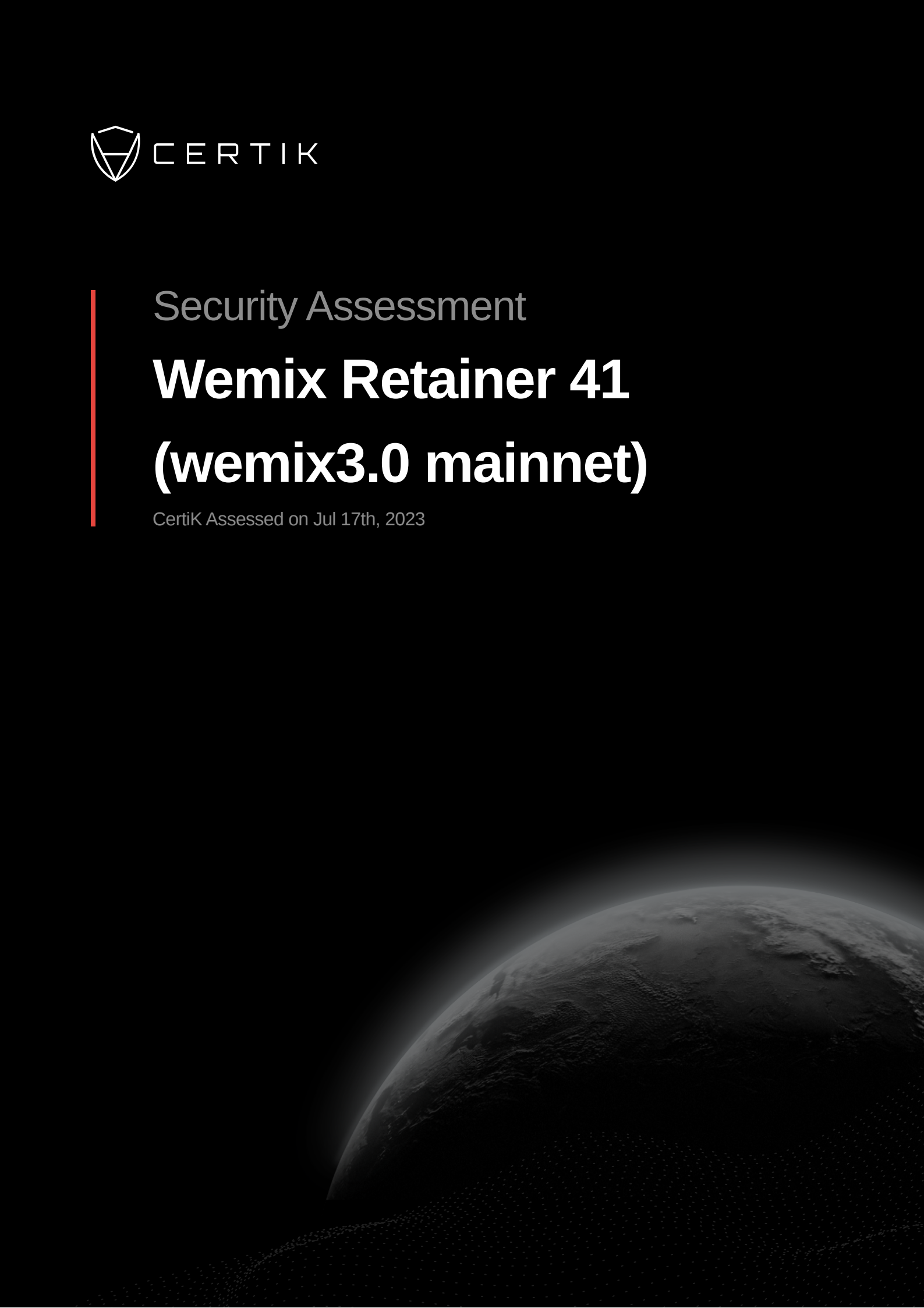
CERTIK

Security Assessment

Wemix Retainer 41 (wemix3.0 mainnet)

CertiK Assessed on Jul 17th, 2023





CertiK Assessed on Jul 17th, 2023

Wemix Retainer 41 (wemix3.0 mainnet)

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
DeFi	Wemix	Manual Review, Static Analysis
LANGUAGE	**TIMELINE**	**KEY COMPONENTS**
Solidity	Delivered on 07/17/2023	N/A

CODEBASE
https://bitbucket.org/wemade-tree/waffle/src/dev/
View All in Codebase Page

COMMITS
5ba6ede2bc8ebaf65c08893f4b05a17934af99e9
View All in Codebase Page

Vulnerability Summary



15	12	0	0	3	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

2	Critical	2 Resolved	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
2	Major	2 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
3	Medium	3 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
4	Minor	4 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
4	Informational	3 Resolved, 1 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | WEMIX RETAINER 41 (WEMIX3.0 MAINNET)

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Findings

NFN-02 : Lack of Validation for `mintInfo`

TTC-01 : Unrestricted Access to Minting Functions

CKP-03 : Centralization Related Risks

NFC-05 : Centralized Control of Contract Upgrade

NFC-01 : Missing validation on refund amount

NFC-02 : Lack of Validation for the target of `authInfo`

PCK-02 : Out of Scope Dependencies

AAA-01 : ` isAliveTreasruy` not reset in function `removeTreasury`

AAA-02 : Incompatibility With Deflationary Tokens

GNG-01 : Lack of Duplicate Check in Function `addList`

NFC-04 : Unchecked Value of ERC-20 `transfer()`/`transferFrom()` Call

AAA-03 : Typo in Variable Name `isAliveTreasruy`

ABD-02 : Missing Emit Events

ERE-01 : Redundant Statements

PCK-03 : Non-standard role naming

Appendix

Disclaimer

CODEBASE | WEMIX RETAINER 41 (WEMIX3.0 MAINNET)

Repository

https://bitbucket.org/wemade-tree/waffle/src/dev/

Commit

5ba6ede2bc8ebaf65c08893f4b05a17934af99e9

AUDIT SCOPE | WEMIX RETAINER 41 (WEMIX3.0 MAINNET)

19 files audited ● 2 files with Acknowledged findings ● 6 files with Resolved findings ● 11 files without findings

ID	File	SHA256 Checksum
● NFC	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol	cbc5c3a18bc11f48085199ec0ed45d5f3a1962ef6fae991f8c4f1d4278ed9197
● ERE	waffle/contracts/Standard/EventReward/EventRewarder.sol	c4c4ba2d4f7bc2f8df348aa357a2b0322071977ae81d89ef916cc8f60a3038ea
● AAA	waffle/Projects/AirDrop/contracts/AirDrop/AirDrop.sol	cfc0c398001e6d179597f5bfcaba46c2c24276942fc021126359f270ca05ede0
● ABD	waffle/Projects/AirDrop/contracts/AirDropBase/AirDropBase.sol	35e75b126a15379f35b3f441ebb03eb4fa82ef37ae29ef96ef0f142939813a76
● GNG	waffle/Projects/ContractDeployer/contracts/GameNFTDeployer/GameNFTDeployer.sol	7587d76937678d39220813e5a8658cbf3feaa9caf83095f011867d4af4933838
● POP	waffle/Projects/ContractDeployer/contracts/ProxyOFT2Deployer/ProxyOFT2Deployer.sol	7eddf1fd6b8af9dd690184e574da23d2ac4fd074426ca33ae9ef10a023a6f4e1
● VMM	waffle/Projects/ContractDeployer/contracts/VaultManagerDeployer/VaultManagerDeployer.sol	330fc17b9d9cf3014e9850f5004468171791330c8a052780c292b48c04180ea0
● TTC	waffle/Projects/TokenCheater/contracts/TokenCheater.sol	40747744cdd035bb1454cd10a3d0c067968607d3a4c40e1fa0f2a7cab9a37b65
● IAA	waffle/Projects/AirDrop/contracts/AirDrop/IAirDrop.sol	d6a70aa2257ec4f52326fa53502dd225023a98dd7dd3d7ee409074b72474b77e
● IAP	waffle/Projects/AirDrop/contracts/AirDropBase/IAirDropBase.sol	850e05f5a10f0f2511c680c666cdce9152632ddb72311c7a41b75dae6e652a86f
● IGT	waffle/Projects/ContractDeployer/contracts/GameNFTDeployer/IGameNFTDeployer.sol	31558c45d82ad0d0e1242388ba9f37639d90aba0b26a0fbac45d628429fb5f68

ID	File	SHA256 Checksum
IDD	waffle/Projects/ContractDeployer/contracts/IDeployer.sol	940f5e43e8159ac9d2a0df2e9227855e97b4752975c59b4bbf028a49b9e08c8d
GFN	waffle/Projects/GameNFT/contracts/NFT/GameNFT.sol	c56542740a56ba587eb0f987463e9a232802e4832f7c9db5d08d56833101616a
IGG	waffle/Projects/GameNFT/contracts/NFT/IGameNFT.sol	721acea67bfe5bb3e718e73974aebbca4005b189e613bd93fd41941b049d80af
INT	waffle/Projects/GameNFT/contracts/NFT/INFTLocker.sol	5ed3f1c97832f6c68e04656670945eac7b61311ed79c0dab92dbf9a5c471e646
NFF	waffle/Projects/GameNFT/contracts/NFT/NFTLocker.sol	98461bf013e26cad04e6ee8e35f9e3e1af94a41500fe94c7fabe3634494aa1ae
INN	waffle/Projects/GameNFT/contracts/Navigator/INFTNavigator.sol	cbdf4216bf5bdba62e44e0ae9f784e38af96d4462e0bcf422fc0374abe008fb6
NFP	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigator.sol	2e55a13e5ba7bbff0ea1eb1e80257f77af2f1698832e387b19eabc383dbd894d
RER	waffle/contracts/Standard/EventReward/Rewarder.sol	52dd788016565817a65129d5aed81e6c8f55a211f974021db5b6e3cbaac4a49a

APPROACH & METHODS

WEMIX RETAINER 41 (WEMIX3.0 MAINNET)

This report has been prepared for Wemix to discover issues and vulnerabilities in the source code of the Wemix Retainer 41 (wemix3.0 mainnet) project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

FINDINGS | WEMIX RETAINER 41 (WEMIX3.0 MAINNET)



15	2	2	3	4	4
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Wemix Retainer 41 (wemix3.0 mainnet). Through this audit, we have uncovered 15 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
NFN-02	Lack Of Validation For `mintInfo`	Logical Issue	Critical	Resolved
TTC-01	Unrestricted Access To Minting Functions	Logical Issue	Critical	Resolved
CKP-03	**Centralization Related Risks**	**Centralization**	**Major**	**Acknowledged**
NFC-05	**Centralized Control Of Contract Upgrade**	**Centralization**	**Major**	**Acknowledged**
NFC-01	Missing Validation On Refund Amount	Logical Issue	Medium	Resolved
NFC-02	Lack Of Validation For The Target Of `authInfo`	Logical Issue	Medium	Resolved
PCK-02	Out Of Scope Dependencies	Logical Issue	Medium	Resolved
AAA-01	`isAliveTreasruy` Not Reset In Function `removeTreasury`	Logical Issue	Minor	Resolved
AAA-02	Incompatibility With Deflationary Tokens	Volatile Code	Minor	Resolved
GNG-01	Lack Of Duplicate Check In Function `addList`	Logical Issue	Minor	Resolved
NFC-04	Unchecked Value Of ERC-20 `transfer()` / `transferFrom()` Call	Volatile Code	Minor	Resolved

ID	Title	Category	Severity	Status
AAA-03	Typo In Variable Name `isAliveTreasruy`	Volatile Code	Informational	Resolved
ABD-02	Missing Emit Events	Coding Style	Informational	Resolved
ERE-01	Redundant Statements	Volatile Code	Informational	Acknowledged
PCK-03	Non-Standard Role Naming	Access Control	Informational	Resolved

NFN-02 | LACK OF VALIDATION FOR `mintInfo`

Category	Severity	Location	Status
Logical Issue	Critical	NFTNavigatorLogic.sol (597a996): 111~112	Resolved

Description

The `mint` function in `NFTNavigatorLogic.sol` and the `collectMintFee` function in `NFTManager.sol` do not validate the `mintInfo` properly.

This could potentially allow a user to mint NFT tokens without any associated costs, by setting `mintInfo.fee` to zero.

```
124     if(!IGameToken(feeToken).transferFrom(authInfo.user, address(nftManager),
 mintInfo.fee)) revert TransferFail(authInfo.user, address(nftManager), mintInfo.fee
);
125         try nftManager.collectMintFee(_msgSender(), mintInfo, authInfo) {
126             return IGameNFT(token).mint(authInfo.user, mintInfo.tokenURI,
 mintInfo.locked);
```

The `collectMintFee` function, in its current state, returns `true` if the `mintInfo.fee` is zero. This means the mint function will still execute and mint the NFT token even when there is no associated fee.

```
function collectMintFee(
      address sender,
      MintInfo calldata mintInfo,
      AuthInfo calldata authInfo
  )
      external
      override
      onlyNavigator
      returns(bool)
  {
      MintInfo memory _mintInfo = mintInfo;
      AuthInfo calldata _authInfo = authInfo;
      address _feeToken = feeToken;

      _checkAuth(sender, _authInfo);

      if(_mintInfo.fee > 0) {
          // income
          IGameToken(_feeToken).increaseAllowance(address(recipientRole), _mintInfo.fee);
          recipientRole.addIncome(_feeToken, service, _authInfo.column, _mintInfo.fee);
      }

      return true;
  }
```

Recommendation

We recommend the team implement proper validation on the input parameter `mintInfo` to ensure users cannot mint NFT tokens at no cost.

Alleviation

Wemix Team:

It is by design that mintInfo.fee can be set to zero. This is to give developers the option to mint NFTs without a fee. However, the user must submit the validator's signature to execute the function, so even if the user arbitrarily sets fee to 0, the function will not be executed. Therefore, if the function is executed with fee set to 0, it can be assumed that the validator intentionally set the value as such.

TTC-01 | UNRESTRICTED ACCESS TO MINTING FUNCTIONS

Category	Severity	Location	Status
Logical Issue	Critical	waffle/Projects/TokenCheater/contracts/TokenCheater.sol: 17~24	Resolved

Description

There is no access control on function `cheatToken` and `cheatItem` , which means anyone can mint tokens or items through these two public functions.

This poses a significant security risk, as a malicious actor could exploit this to inflate the supply of tokens or NFT items, potentially devaluing the user assets and damage the whole tokenomics.

```
 17      function cheatToken(address token, bytes32 service, address to, uint amount
) external {
 18          require(GTNavigator.mint(token, service, to, amount),
"TokenCheater: cheatToken error");
 19      }
 20
 21      //cheat item
 22      function cheatItem(address token, address to, string memory tokenURI, bool
 locked) external {
 23          require(NFTNaviator.bulkMint(token, to, tokenURI, locked),
"TokenCheater: cheatItem error");
 24      }
```

Recommendation

We recommend enforcing proper access control on the aforementioned functions.

Alleviation

Wemix Team:

This contract is only deployed in a test network environment and was developed to support the faucet function within the test environment.

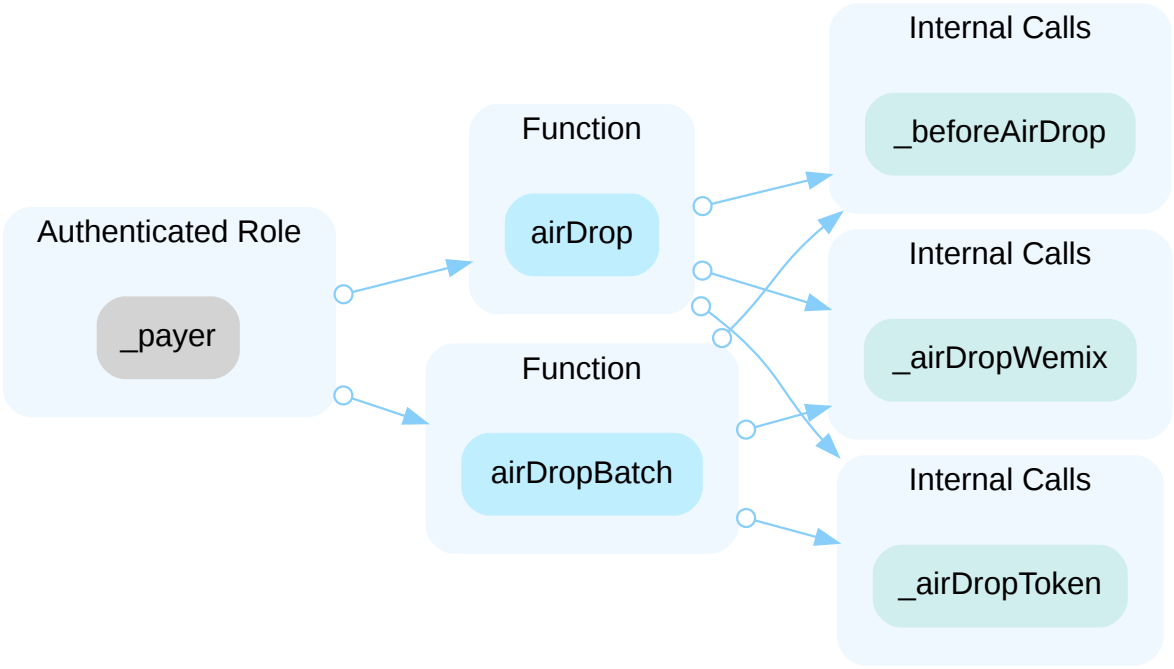
It will never be deployed to the main network, and if we urgently need to stop the cheat function for tokens and items, we can delete the mint role of FTs and NFTs that this contract has. The validation for the role is defined in the mint() of FTs and NFTs, not in this contract.

CKP-03 | CENTRALIZATION RELATED RISKS

Category	Severity	Location	Status
Centralization	Major	waffle/contracts/Standard/EventReward/EventRewarder.sol: 100~127; AirDrop.sol (597a996): 47, 78, 112, 153, 183, 209, 240, 265, 296, 313; NFTNavigatorLogic.sol (597a996): 108, 145, 171, 213, 240, 263, 280, 297, 314, 377; Rewarder.sol (597a996): 61, 78, 85, 89, 93	Acknowledged

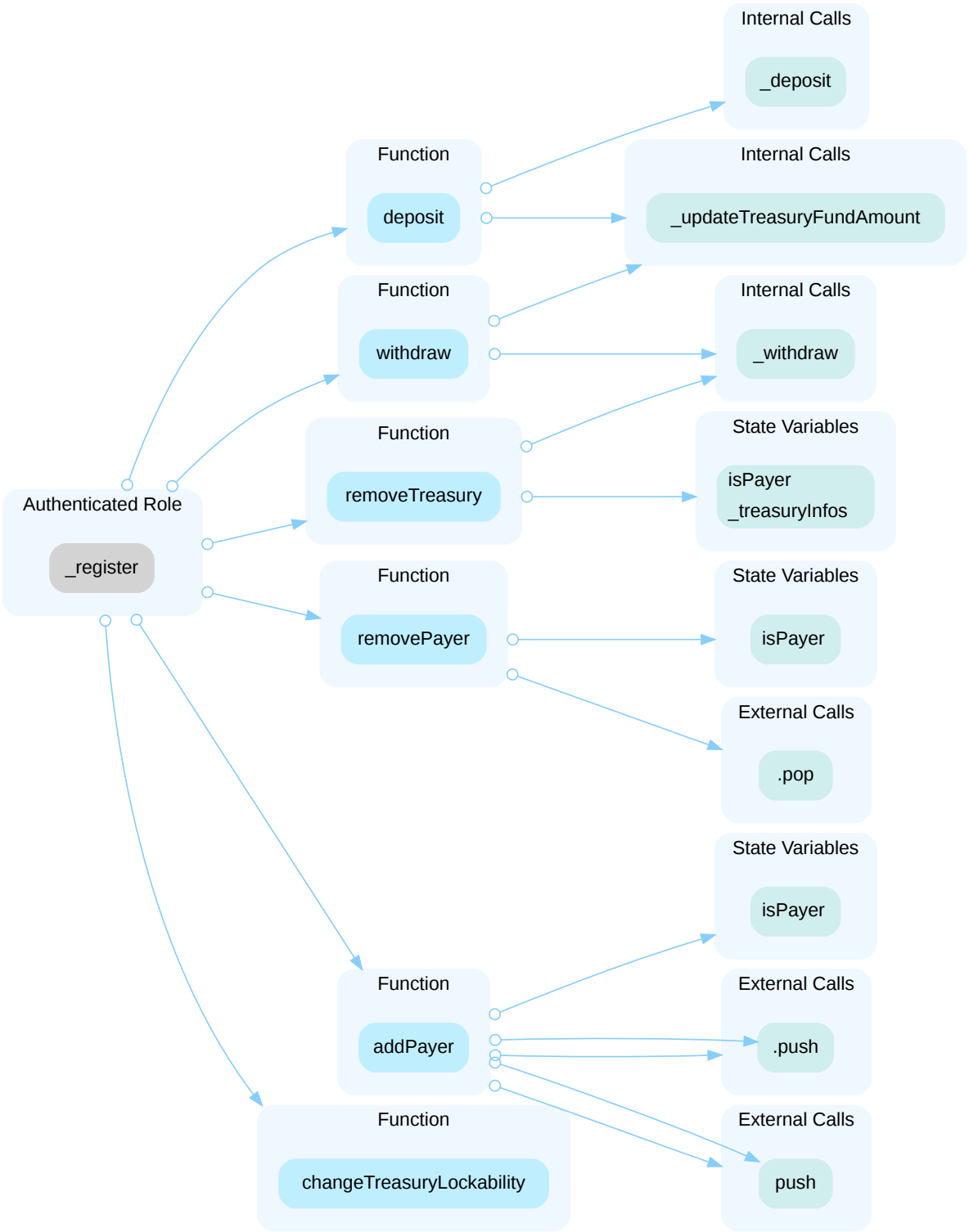
Description

In the contract `AirDrop` the role `_payer` has authority over the functions shown in the diagram below. Any compromise to the `_payer` account may allow the hacker to take advantage of this authority and airdrop treasury fund to arbitrary accounts.

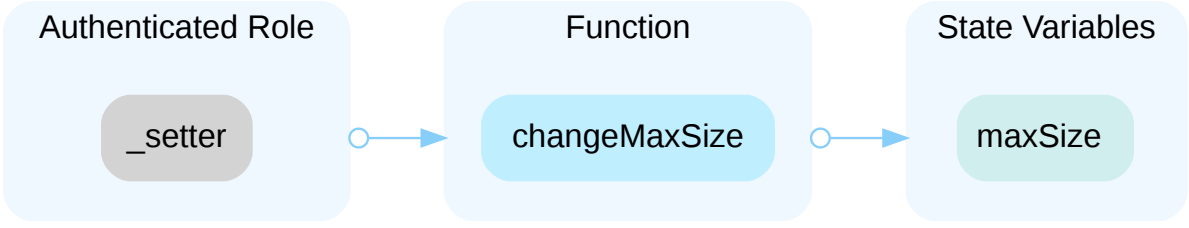


In the contract `AirDrop` the role `_register` has authority over the functions shown in the diagram below. Any compromise to the `_register` account may allow the hacker to take advantage of this authority and

- withdraw funds from treasury
- add/remove payer role.
- change treasury lockability



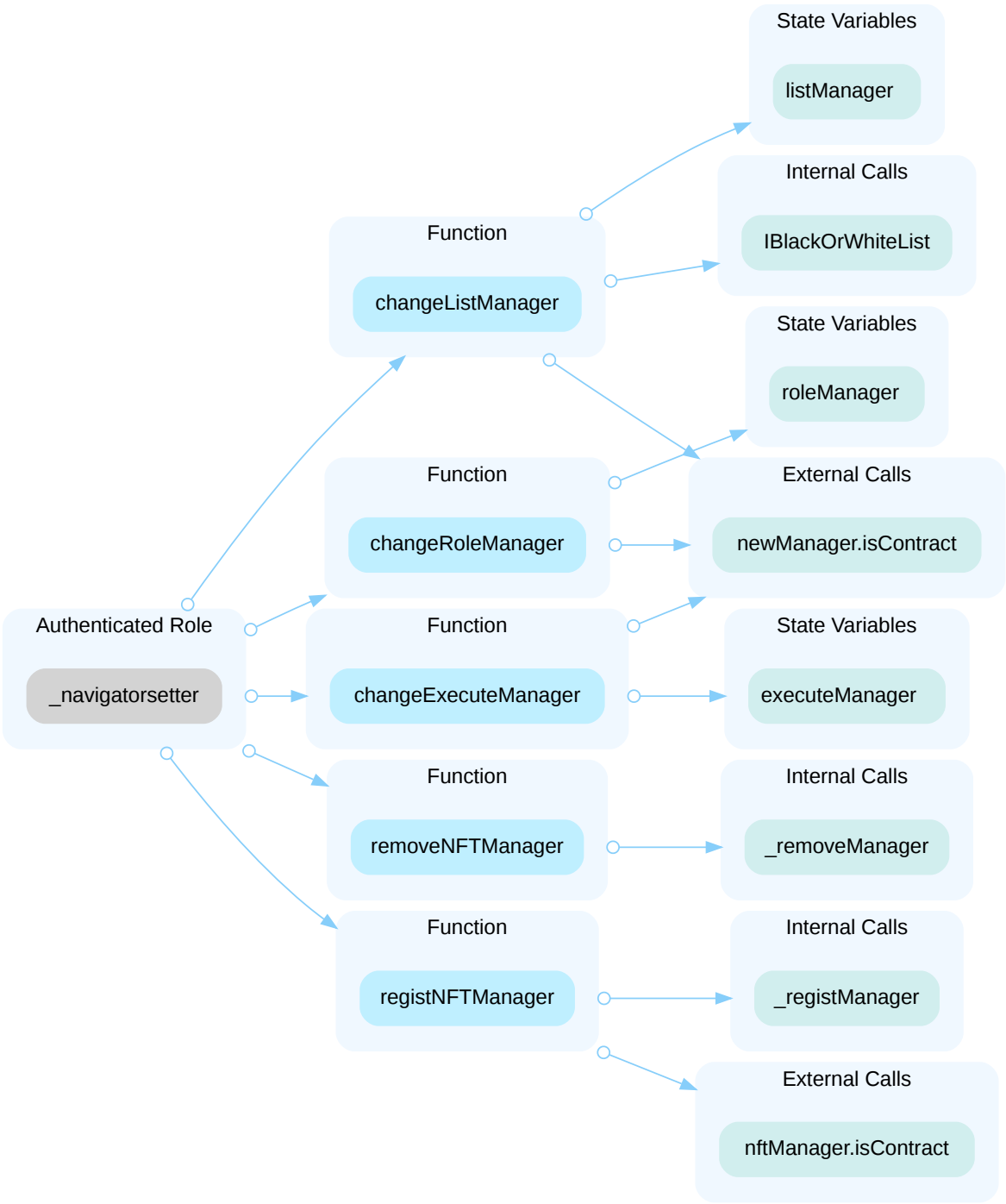
In the contract `AirDrop` the role `_setter` has authority over the functions shown in the diagram below. Any compromise to the `_setter` account may allow the hacker to take advantage of this authority and modify the `maxSize`.



In the contract `NFTNavigatorLogic` the role `_minter` has authority over the functions shown in the diagram below. Any compromise to the `_minter` account may allow the hacker to take advantage of this authority and mint NFT tokens to any account.

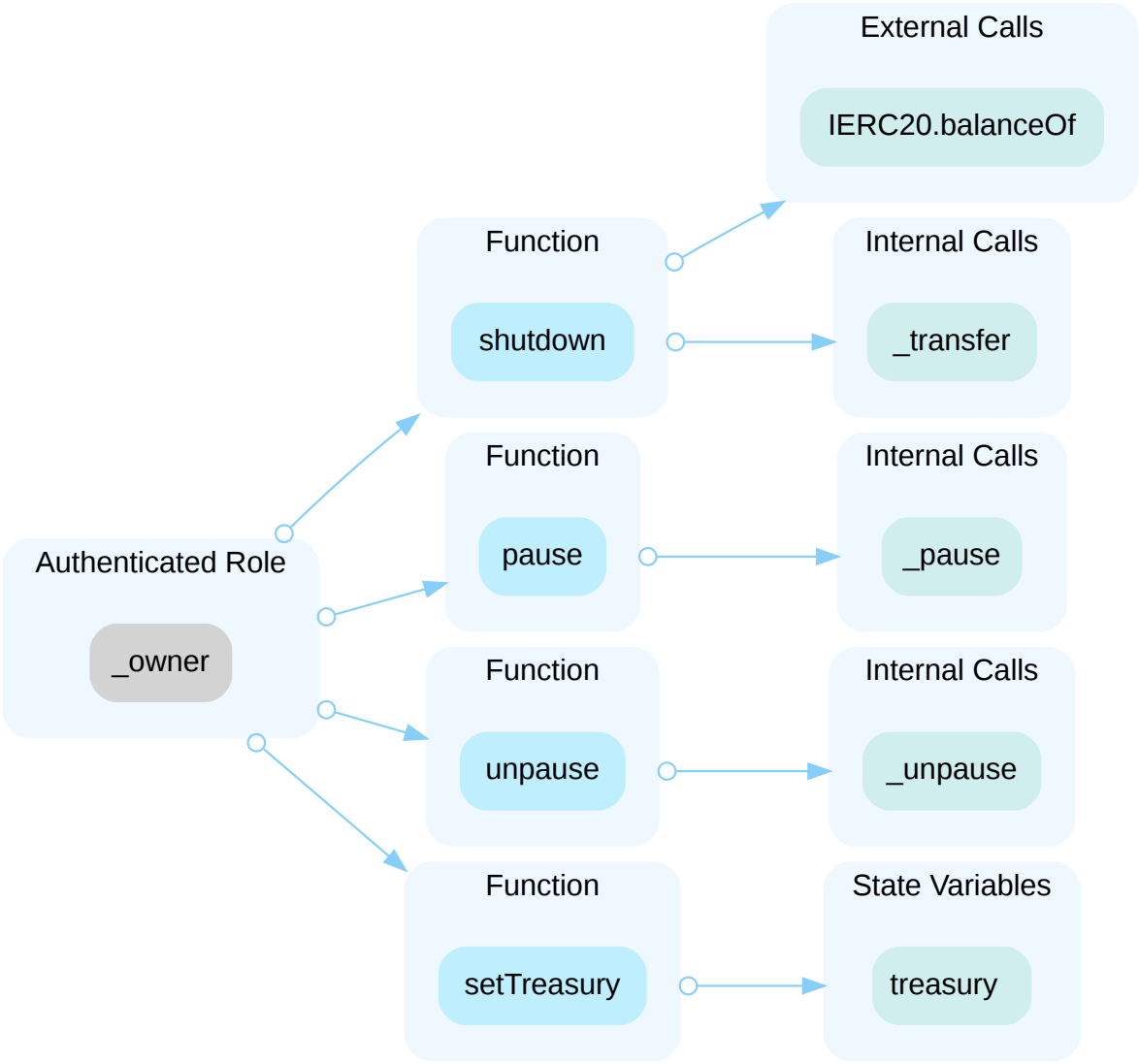


In the contract `NFTNavigatorLogic` the role `_navigatorsetter` has authority over the functions shown in the diagram below. Any compromise to the `_navigatorsetter` account may allow the hacker to take advantage of this authority and change manager contract addresses.



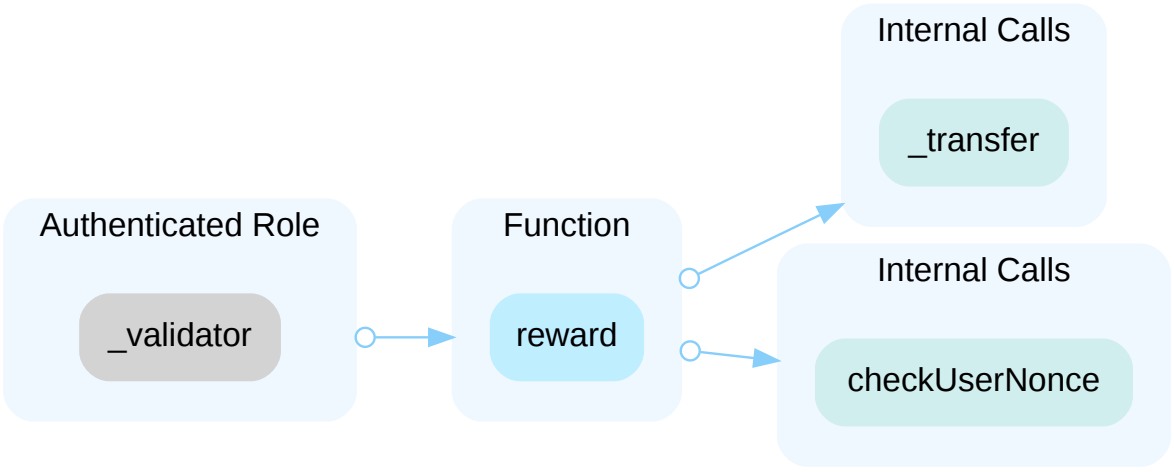
In the contract `Rewarder` the role `_owner` has authority over the functions shown in the diagram below.

Any compromise to the `_owner` account may allow the hacker to take advantage of this authority and emergency withdraw all funds from the treasury through the function `shutdown`.



In the contract `EventRewarder` the role `_validator` has authority over the functions shown in the diagram below.

Any compromise to the `_validator` account may allow the hacker to take advantage of this authority and transfer `rt` token to arbitrary account through the function `reward`.



Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
  OR
- Remove the risky functionality.

Alleviation

Wemix Team:

Minter permissions are separated per NFT, so if minter permissions are taken for one NFT, they cannot be abused for other NFTs. Furthermore, the minter permissions of an attacked NFT can be revoked by an account with higher permissions, and in the worst case scenario, the functions that can be performed through minter can be controlled, including a complete pause.

NFC-05 | CENTRALIZED CONTROL OF CONTRACT UPGRADE

Category	Severity	Location	Status
Centralization	Major	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol	Acknowledged

Description

In the contract `NFTNavigatorLogic` , the role `defaultOwner` has the authority to update the implementation contract behind the `NFTNavigator` contract.

Any compromise to the `defaultOwner` account may allow a hacker to take advantage of this authority and change the implementation contract which is pointed by proxy and therefore execute potential malicious functionality in the implementation contract.

Recommendation

We recommend that the team make efforts to restrict access to the admin of the proxy contract. A strategy of combining a time-lock and a multi-signature (⅔, ⅗) wallet can be used to prevent a single point of failure due to a private key compromise. In addition, the team should be transparent and notify the community in advance whenever they plan to migrate to a new implementation contract.

Here are some feasible short-term and long-term suggestions that would mitigate the potential risk to a different level and suggestions that would permanently fully resolve the risk.

Short Term:

A combination of a time-lock and a multi signature (⅔, ⅗) wallet mitigate the risk by delaying the sensitive operation and avoiding a single point of key management failure.

- A time-lock with reasonable latency, such as 48 hours, for awareness of privileged operations;
  AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to a private key compromised;
  AND
- A medium/blog link for sharing the time-lock contract and multi-signers addresses information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.

- Provide a link to the **medium/blog** with all of the above information included.

Long Term:

A combination of a time-lock on the contract upgrade operation and a DAO for controlling the upgrade operation mitigate the contract upgrade risk by applying transparency and decentralization.

- A time-lock with reasonable latency, such as 48 hours, for community awareness of privileged operations;
  AND
- Introduction of a DAO, governance, or voting module to increase decentralization, transparency, and user involvement;
  AND
- A medium/blog link for sharing the time-lock contract, multi-signers addresses, and DAO information with the community.

For remediation and mitigated status, please provide the following information:

- Provide the deployed time-lock address.
- Provide the **gnosis** address with **ALL** the multi-signer addresses for the verification process.
- Provide a link to the **medium/blog** with all of the above information included.

Permanent:

Renouncing ownership of the `admin` account or removing the upgrade functionality can *fully* resolve the risk.

- Renounce the ownership and never claim back the privileged role;
  OR
- Remove the risky functionality.

Note: we recommend the project team consider the long-term solution or the permanent solution. The project team shall make a decision based on the current state of their project, timeline, and project resources.

Alleviation

Wemix Team:

The defaultOwner's account is controlled by a 2 of 3 multisig wallet.

NFC-01 | MISSING VALIDATION ON REFUND AMOUNT

Category	Severity	Location	Status
Logical Issue	Medium	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol: 190~194	Resolved

Description

In the current implementation, the `burn` function allows for a refund mechanism where an amount of tokens is transferred from the vault to a user's account. However, the function does not include any validation checks to ensure that the amount to be refunded is valid. Without proper checks, users can pick arbitrary refund amount which could drain the feeToken balance of the vault contract.

```
171     function burn(
172         address token,
173         bytes32 service,
174         uint tokenId,
175         uint amount,
176         AuthInfo calldata authInfo
177     )
178         external
179         isBlacklist
180         onlyExecutable(INFTNavigator.burn.selector)
181         returns(bool)
182     {
183         INFTManager nftManager = INFTManager(_getNFTManager(token, service));
184         address feeToken = INFTManager(nftManager).feeToken();
185         address vault = INFTManager(nftManager).vault();
186 
187         _executeTx(nftManager.checkAuth, authInfo);
188 
189         try IGameNFT(token).burn(authInfo.user, tokenId) {
190             if (amount > 0) {
191                 emit Refunded(authInfo.user, amount, authInfo.column, block.
number);
192 
193                 return IGameToken(feeToken).transferFrom(vault, authInfo.user,
 amount);
194             }
195             return true;
196         ...
```

Recommendation

We recommend the team implement proper validation check for the refund `amount` prior to initiating the `transferFrom` function call.

Alleviation

Wemix Team:

A signature from an authorized validator is required to perform the burn function. This is defined by the field value of authInfo among the parameters being entered. Also, on our platform, the validator signature is designed to be entered last, after all other parameters have been entered. This means that when a validator's signature is submitted to perform a burn function, it means that the validator has verified the parameters entered in that function. So, no matter what number amount was entered (even if it was entered as 0), if the validator generated a signature and executed the function, it means as a matter of policy that the value was the correct parameter that was verified by the validator. The responsibility for this lies with the validator who submitted the signature without correctly pre-validating the relevant parameters.

CertiK Team:

The team has incorporated a selector value corresponding to the function they aim to invoke during the generation of the verifyKey value, a critical component for user signature creation. Changes have been reflected in this commit.

NFC-02 | LACK OF VALIDATION FOR THE TARGET OF `authInfo`

Category	Severity	Location	Status
Logical Issue	Medium	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol: 171~177, 213~218	Resolved

Description

In the current implementation, the function call `checkAuth` of `authInfo` in contract `nftManager` does not validate function arguments `token` and `tokenId` properly. This could lead to misuse of the provided authorization.

Let's say if a user obtains valid authInfo from the validator role, there are no checks in place to confirm what this authorization is actually granted for. Therefore, the user could potentially use this single authorization for multiple purposes, such as minting/burning NFT tokens or changing the status of NFT tokens, which may not have been the intended permission.

Recommendation

Since the `nftManager` is out of this audit scope, the auditor would like to recommend the team to perform a thorough review of the `nftManager` contract, specifically focusing on the implementation of the `authInfo` validation. The current `burn` function and `changeStatus` function rely heavily on this validation, and any potential vulnerabilities in the `nftManager` could directly impact the security of the navigator contract.

Alleviation

Wemix Team:

To address this issue, we now add a selector value for the function we want to call when generating the verifyKey value needed to generate the user signature. Therefore, if a signature submitted by a user to execute function A() is used by a malicious validator to execute B(), the signature will be determined to be invalid.

Changes have been reflected in this commit.

PCK-02 | OUT OF SCOPE DEPENDENCIES

Category	Severity	Location	Status
Logical Issue	Medium	waffle/Projects/AirDrop/contracts/AirDropBase/AirDropBase.sol: 39~40; waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol: 20~22	Resolved

Description

The `NFTNavigatorLogic` and `AirdropBase` contracts have several dependencies which are not part of the audit scope, namely `roleManager`, `executeManager`, `nftManager`, and `listManager`. These dependencies appear to manage access control, execution management, NFT token management, and blacklist/whitelist functionality respectively.

```
20      roleManager = IAccessControl(_roleManager);
21      executeManager = IExecuteManager(_executeManager);
22      IBlackOrWhiteList public listManager;
```

The scope of the audit treats contract that is out of scope as black boxes and assumes their functional correctness.

While the auditing of `NFTNavigatorLogic` and `AirdropBase` contract might not reveal any inherent vulnerabilities, the security of the contract is heavily influenced by these external dependencies. As such, vulnerabilities or misconfigurations in these dependencies could potentially compromise the security of the audited contract.

Recommendation

The aforementioned contracts are out of the audit scope. We encourage the team to constantly monitor the status of the those contracts and ensure its security and functionality correctness.

Alleviation

Wemix Team:

These contracts are involved in another audit request and have already been audited.

AAA-01 `isAliveTreasruy` NOT RESET IN FUNCTION `removeTreasury`

Category	Severity	Location	Status
Logical Issue	Minor	waffle/Projects/AirDrop/contracts/AirDrop/AirDrop.sol: 153~176	Resolved

Description

In the current implementation, the contract uses a mapping `isAliveTreasruy` to keep track of whether a treasury is alive or not, meaning it's active and hasn't been removed. This is set to true within the createTreasury function when a new treasury is created.

However, in the removeTreasury function, there's no corresponding operation to set `isAliveTreasruy[treasuryId]` to false or delete the treasuryId from the `isAliveTreasruy` mapping. This omission could cause a logical inconsistency in the state of the contract, as a removed treasury would still be marked as alive. This could lead to potential issues in subsequent contract operations.

Recommendation

We recommend setting the `isAliveTreasruy[treasuryId]` to false or delete the treasuryId from the `isAliveTreasruy` mapping in function `removeTreasury`.

Alleviation

The team heeded our advice and fixed the issue in this commit.

AAA-02 | INCOMPATIBILITY WITH DEFLATIONARY TOKENS

Category	Severity	Location	Status
Volatile Code	Minor	waffle/Projects/AirDrop/contracts/AirDrop/AirDrop.sol: 112	Resolved

Description

When transferring standard ERC20 deflationary tokens, the input amount may not be equal to the received amount due to the charged transaction fee. For example, if a register deposits 100 deflationary tokens (with a 10% transaction fee) in an Airdrop contract, only 90 tokens actually arrive in the contract. If the register role invokes `withdraw(id,0)` to withdraw all unused token amount, that call will revert because of the mismatch between the actual balance and pending balance.

Recommendation

We recommend regulating the set of ERC-20 tokens supported and adding necessary mitigation mechanisms to keep track of accurate balances, if there is a need to support deflationary tokens.

Alleviation

The client heeded our advice and added new code logic to check the balance of treasury to support deflationary tokens. Changes have been reflected in this commit.

GNG-01 | LACK OF DUPLICATE CHECK IN FUNCTION `addList`

Category	Severity	Location	Status
Logical Issue	Minor	waffle/Projects/ContractDeployer/contracts/GameNFTDeployer/GameNFTDeployer.sol: 63~70	Resolved

Description

The addList function in the contract allows the owner to add new entries to the `registerList` array. However, there are no checks to prevent duplicate entries from being added.

```
63      function addList(RegistList calldata list) external onlyOwner {
64          RegistList[] storage _registerList = registList;
65          if(list.to == address(0)) revert InvalidAddress(list.to);
66
67          _registerList.push(list);
68
69          emit ListAdded(list, _registerList.length - 1);
70      }
```

This lack of validation could potentially lead to wastage of storage and could also cause logical errors in other parts of the contract if they are not designed to handle duplicate entries correctly.

Recommendation

We recommend the team add proper validation to the `addList` function to prevent duplicate entries.

Alleviation

The team heeded our advice and fixed the issue in this commit.

NFC-04 UNCHECKED VALUE OF ERC-20 `transfer()` / `transferFrom()` CALL

Category	Severity	Location	Status
Volatile Code	Minor	waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol: 191~193	Resolved

Description

The function `IGameToken(feeToken).transferFrom` is used without checking its return value.

If this function fails silently (returns false), the contract would still emit the Refunded event, leading to an inconsistency between the contract's state and emitted events.

```
189     try IGameNFT(token).burn(authInfo.user, tokenId) {
190             if (amount > 0) {
191                 emit Refunded(authInfo.user, amount, authInfo.column, block.
number);
192
193                 return IGameToken(feeToken).transferFrom(vault, authInfo.user,
 amount);
194             }
195             return true;
```

Recommendation

We recommend only emit event when the linked `transferFrom` call successfully executes (returns true).

Alleviation

The team heeded our advice and fixed the issue in this commit.

AAA-03 | TYPO IN VARIABLE NAME `isAliveTreasruy`

Category	Severity	Location	Status
Volatile Code	Informational	waffle/Projects/AirDrop/contracts/AirDrop/AirDrop.sol: 18	Resolved

Description

There appears to be a typographical error in the naming of the `isAliveTreasruy` variable in the provided Solidity code. Presumably, the intended name was `isAliveTreasury`, which better reflects the function of this variable.

```
18    mapping(bytes32 => bool) public isAliveTreasruy;
```

Recommendation

Consider renaming `isAliveTreasruy` to `isAliveTreasury`.

Alleviation

The team heeded our advice and fixed the issue in this commit.

ABD-02 | MISSING EMIT EVENTS

Category	Severity	Location	Status
Coding Style	Informational	waffle/Projects/AirDrop/contracts/AirDropBase/AirDropBase.sol: 48~62	Resolved

Description

Functions that update state variables should emit relevant events as notifications.

```
48      function changeRoleManager(address newManager) external onlySetter {
49          if(!newManager.isContract()) revert InvalidAddress(newManager);
50
51          roleManager = IAccessControl(newManager);
52      }
53
54      function changeExecuteManager(address newManager) external onlySetter {
55          if(!newManager.isContract()) revert InvalidAddress(newManager);
56
57          executeManager = IExecuteManager(newManager);
58      }
59
```

Recommendation

We recommend adding events for state-changing actions, and emitting them in their relevant functions.

Alleviation

The team heeded our advice and fixed the issue in this commit.

ERE-01 REDUNDANT STATEMENTS

Category	Severity	Location	Status
Volatile Code	Informational	waffle/contracts/Standard/EventReward/EventRewarder.sol: 74~75	Acknowledged

Description

The child contract constructor is repeating some of the operations that are already performed in the parent contract's constructor. Specifically, the assignments of the rt and treasury variables are redundant because they are already carried out in the parent constructor.

Code redundancy may lead to wastage of gas, as unnecessary operations are performed.

```
74          rt = token;
75          treasury = address(this);
```

Recommendation

We recommend remove the redundant lines `rt = token` and `treasury = address(this)` from the child contract's constructor.

PCK-03 | NON-STANDARD ROLE NAMING

Category	Severity	Location	Status
Access Control	Informational	waffle/Projects/ContractDeployer/contracts/GameNFTDeployer/GameNFTDeployer.sol: 13~14; waffle/Projects/ContractDeployer/contracts/ProxyOFT2Deployer/ProxyOFT2Deployer.sol: 18~19; waffle/Projects/ContractDeployer/contracts/VaultManagerDeployer/VaultManagerDeployer.sol: 24~26; waffle/Projects/GameNFT/contracts/Navigator/NFTNavigatorLogic.sol: 96~97	Resolved

Description

The provided code assigns the name of the role "defaultOwner" as "0x00". This is not recommended and could lead to confusion.

According to OpenZeppelin's standard guidelines, roles should be referred to by their unique bytes32 identifiers:

Roles are referred to by their bytes32 identifier. These should be exposed in the external API and be unique. The best way to achieve this is by using public constant hash digests

```
1    bytes32 public constant MY_ROLE = keccak256("MY_ROLE");
```

Recommendation

Consider changing the assignment of "defaultOwner" from "0x00" to a unique bytes32 hash value, following the OpenZeppelin's standard guidelines. For instance:

```
1    bytes32 public constant DEFAULT_OWNER_ROLE = keccak256("DEFAULT_OWNER_ROLE");
```

Alleviation

Wemix Team:

We inherit AccessControl contract made by OpenZeppelin. AccessControl define DEFAULT_ADMIN_ROLE is "0x00". "defaultOwner" and DEFAULT_ADMIN_ROLE both have the same role.

APPENDIX | WEMIX RETAINER 41 (WEMIX3.0 MAINNET)

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Access Control	Access Control findings are about security vulnerabilities that make protected assets unsafe.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

